

# 8 – EXERCISES ON MATRICES

## Quick reference

---

```
#include <array>
#include <vector>

// std::array: fixed size, known at compile time
std::array<int, 4> notes = {60, 62, 64, 65};
notes[2]      = 70;          // access by index
notes.size() == 4;          // always 4

// std::vector: dynamic size, can grow at runtime
std::vector<int> track = {80, 90, 70};
track.push_back(100);       // now has 4 elements
track.size() == 4;
track[0]      == 80;

// 2D array (fixed rows and columns)
std::array<std::array<int, 3>, 2> grid = {{{10, 20, 30},
                                           {40, 50, 60}}};
grid[1][2] == 60;           // row 1, column 2

// 2D vector (dynamic rows and columns)
std::vector<std::vector<int>> image;
image.push_back({255, 0, 128});
image.push_back({64, 32, 16});
image[0][1] == 0;

// Nested loop over a 2D container
for (int row = 0; row < image.size(); row = row + 1) {
    for (int col = 0; col < image[row].size(); col = col + 1) {
        // use image[row][col]
    }
}
```

---

## Part A — Matrix shape and indexing

In this part, indexing is done with numbers, and with the convention (row number, column number). Same goes for the shape.

### Problem 1 — Shaping

Fill in Table 1 according to the following matrices.

| Matrix A                                                                                                  |      | Matrix B | Matrix C |      |    |   |   | Matrix D |                                                                                           |  | Matrix E |  |  |      |      |      |      |      |                                                                                                                 |  |  |     |      |       |      |      |      |                                                                             |  |    |   |    |   |
|-----------------------------------------------------------------------------------------------------------|------|----------|----------|------|----|---|---|----------|-------------------------------------------------------------------------------------------|--|----------|--|--|------|------|------|------|------|-----------------------------------------------------------------------------------------------------------------|--|--|-----|------|-------|------|------|------|-----------------------------------------------------------------------------|--|----|---|----|---|
| <table><tr><td>42</td><td>7</td></tr><tr><td>10</td><td>99</td></tr><tr><td>5</td><td>1</td></tr></table> |      | 42       | 7        | 10   | 99 | 5 | 1 | Name     | <table><tr><td>0.33</td><td>4.20</td><td>9.88</td><td>1.00</td><td>5.55</td></tr></table> |  |          |  |  | 0.33 | 4.20 | 9.88 | 1.00 | 5.55 | <table><tr><td>Red</td><td>Blue</td><td>Green</td></tr><tr><td>Pink</td><td>Cyan</td><td>Lime</td></tr></table> |  |  | Red | Blue | Green | Pink | Cyan | Lime | <table><tr><td>10</td><td>5</td></tr><tr><td>20</td><td>8</td></tr></table> |  | 10 | 5 | 20 | 8 |
|                                                                                                           |      | 42       | 7        |      |    |   |   |          |                                                                                           |  |          |  |  |      |      |      |      |      |                                                                                                                 |  |  |     |      |       |      |      |      |                                                                             |  |    |   |    |   |
|                                                                                                           |      | 10       | 99       |      |    |   |   |          |                                                                                           |  |          |  |  |      |      |      |      |      |                                                                                                                 |  |  |     |      |       |      |      |      |                                                                             |  |    |   |    |   |
|                                                                                                           |      | 5        | 1        |      |    |   |   |          |                                                                                           |  |          |  |  |      |      |      |      |      |                                                                                                                 |  |  |     |      |       |      |      |      |                                                                             |  |    |   |    |   |
| 0.33                                                                                                      | 4.20 | 9.88     | 1.00     | 5.55 |    |   |   |          |                                                                                           |  |          |  |  |      |      |      |      |      |                                                                                                                 |  |  |     |      |       |      |      |      |                                                                             |  |    |   |    |   |
| Red                                                                                                       | Blue | Green    |          |      |    |   |   |          |                                                                                           |  |          |  |  |      |      |      |      |      |                                                                                                                 |  |  |     |      |       |      |      |      |                                                                             |  |    |   |    |   |
| Pink                                                                                                      | Cyan | Lime     |          |      |    |   |   |          |                                                                                           |  |          |  |  |      |      |      |      |      |                                                                                                                 |  |  |     |      |       |      |      |      |                                                                             |  |    |   |    |   |
| 10                                                                                                        | 5    |          |          |      |    |   |   |          |                                                                                           |  |          |  |  |      |      |      |      |      |                                                                                                                 |  |  |     |      |       |      |      |      |                                                                             |  |    |   |    |   |
| 20                                                                                                        | 8    |          |          |      |    |   |   |          |                                                                                           |  |          |  |  |      |      |      |      |      |                                                                                                                 |  |  |     |      |       |      |      |      |                                                                             |  |    |   |    |   |
| User                                                                                                      |      |          |          |      |    |   |   |          |                                                                                           |  |          |  |  |      |      |      |      |      |                                                                                                                 |  |  |     |      |       |      |      |      |                                                                             |  |    |   |    |   |
| Admin                                                                                                     |      |          |          |      |    |   |   |          |                                                                                           |  |          |  |  |      |      |      |      |      |                                                                                                                 |  |  |     |      |       |      |      |      |                                                                             |  |    |   |    |   |
| Guest                                                                                                     |      |          |          |      |    |   |   |          |                                                                                           |  |          |  |  |      |      |      |      |      |                                                                                                                 |  |  |     |      |       |      |      |      |                                                                             |  |    |   |    |   |

Table 1

| Matrix | Number of rows | Number of columns | Shape |
|--------|----------------|-------------------|-------|
| A      |                |                   |       |
| B      |                |                   |       |
| C      |                |                   |       |
| D      |                |                   |       |
| E      |                |                   |       |

### Problem 2 — Indexing

Referring to the matrices of the previous exercise, answer the following questions:

1. Element at index (2, 2) of matrix A:
2. Element at index (3, 1) of matrix B:
3. Index of value 5 in matrix A:
4. Index of value 1.00 in matrix C:
5. Element at index (2, 3) of matrix D:
6. Element at index (2, 1) of matrix E:
7. Index of value Cyan in matrix D:
8. Index of value 8 in matrix E:

## Part B Manipulating spreadsheets

### Problem 1 — Image channel manipulation

Open the file `landscape.xlsx` with o-spreadsheet.

- Go to the sheet “red”
- Set the value of A1 to 0
- Select and drag the bottom right corner of the A1 cell until the cell AG1
- While keeping the complete line selected, drag the bottom right corner of AG1 until AG33
- You have now filled the red channel of the landscape image with 0s
- Save the spreadsheet as a new .xlsx file
- Use the program `image_exporter` to export the .xlsx file to a .png file as illustrated on Figure 1
- Reproduce the experience for the channel green and the channel blue.

Figure 1: Using `image_exporter`

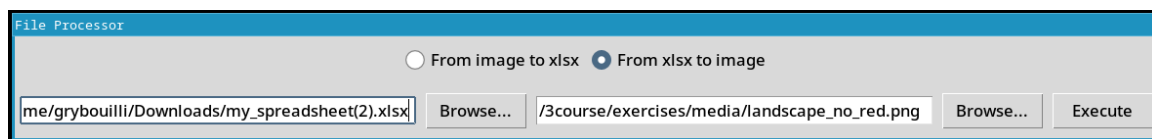
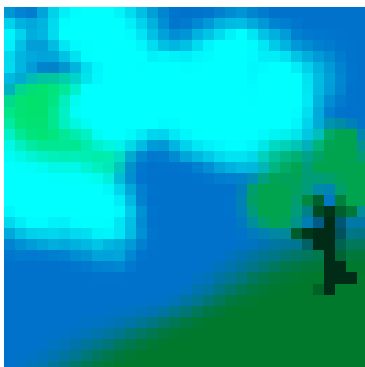
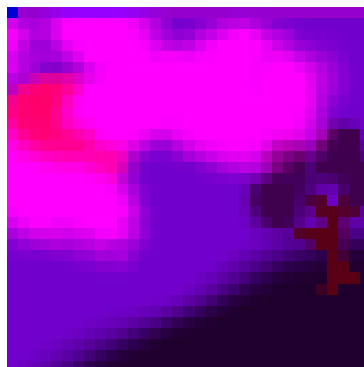


Figure 2: Landscape images with zero'ed channels

(a) Landscape image with zero'ed red channel



(b) Landscape image with zero'ed green channel



(c) Landscape image with zero'ed blue channel



## Part C — `std::array`

### Problem 1 — Trace the output

What does this program print?

Scale degrees

---

```
1  #include <iostream>
2  #include <array>
3
4  int main() {
5      std::array<int, 5> scale = {0, 2, 4, 5, 7};
6
7      std::cout << scale[0]    << std::endl; // line A
8      std::cout << scale[4]    << std::endl; // line B
9      std::cout << scale.size() << std::endl; // line C
10
11     scale[2] = scale[2] + 1;
12     std::cout << scale[2]    << std::endl; // line D
13
14     int total = 0;
15     for (int i = 0; i < scale.size(); i = i + 1)
16         total = total + scale[i];
17     std::cout << total        << std::endl; // line E
18
19     return 0;
20 }
```

---

Line A:

Line B:

Line C:

Line D:

Line E:

### Problem 2 — Spot the bug

This function should return the largest value in an array of five brightness values. It contains one bug. What is it? Try and propose a fix.

Buggy maximum

---

```
1  #include <array>
```

```

2
3 int max_brightness(std::array<int, 5> values) {
4     int max = 0;
5     for (int i = 1; i <= values.size(); i = i + 1) {
6         if (values[i] > max)
7             max = values[i];
8     }
9     return max;
10 }

```

---

### Bug description:

Use: <https://godbolt.org/z/E9T8cGzY7>

### Link to your fix:

*Hint: What happens on the last iteration when  $i == 5$ ?*

## Problem 3 — Write the code

Write a function:

---

```
int sum(const std::array<int, 6>& values);
```

---

that returns the sum of all elements in `values`. Then write a `main` that declares:

---

```
std::array<int, 6> volumes = {70, 85, 90, 60, 75, 80};
```

---

prints the sum, then prints the average (integer division).

### Expected output:

```

460
76

```

## Part D — `std::vector`

### Problem 1 — Trace the output

What does this program print?

```
1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      std::vector<int> durations;
6
7      durations.push_back(210);
8      durations.push_back(185);
9      durations.push_back(240);
10
11     std::cout << durations.size() << std::endl; // line A
12     std::cout << durations[1] << std::endl; // line B
13
14     durations[0] = durations[0] + 15;
15     std::cout << durations[0] << std::endl; // line C
16
17     durations.push_back(durations[2] - 10);
18     std::cout << durations.size() << std::endl; // line D
19     std::cout << durations[3] << std::endl; // line E
20
21     return 0;
22 }
```

---

Line A:

Line B:

Line C:

Line D:

Line E:

## Problem 2 — What does this return?

### Vector accumulation

---

```
1  #include <vector>
2
3  int count_above(const std::vector<int>& values, int threshold) {
4      int count = 0;
5      for (int i = 0; i < values.size(); i = i + 1) {
6          if (values[i] > threshold)
7              count = count + 1;
8      }
9      return count;
10 }
11
12 int main() {
13     std::vector<int> pixels = {30, 180, 90, 220, 15, 200, 110};
14     return count_above(pixels, 100);
15 }
```

---

#### Return value:

*Modify the code to list the elements that are above the threshold and write them here:*

## Problem 3 — Write the code

Write a complete program that:

1. Reads an arbitrary number of integer scores from `argv` (one per argument).
2. Stores them in a `std::vector<int>`.
3. Prints the minimum and maximum values found.

If no arguments are given, print `"no scores"` and return 1.

**Expected output for** `./scores 40 95 72 18 88`:

```
min: 18
max: 95
```

## Part E — 2D `std::array`

### Problem 1 — Trace the output

What does this program print?

2x4 image

---

```
1 #include <iostream>
2 #include <array>
3
4 int main() {
5     std::array<std::array<int, 4>, 2> image = {{
6         {10, 50, 90, 130},
7         {20, 60, 100, 140}
8     }};
9
10    std::cout << image[0][0] << std::endl; // line A
11    std::cout << image[1][3] << std::endl; // line B
12    std::cout << image[0][2] + image[1][1] << std::endl; // line C
13
14    int total = 0;
15    for (int row = 0; row < 2; row = row + 1)
16        for (int col = 0; col < 4; col = col + 1)
17            total = total + image[row][col];
18    std::cout << total << std::endl; // line D
19
20    return 0;
21 }
```

---

Line A:

Line B:

Line C:

Line D:

### Problem 2 — Write the code

Write a function:

---

```
void print_grid(const std::array<std::array<int, 4>, 3>& grid);
```

---

that prints each row on its own line, values separated by spaces. Then write a `main` that declares the following grid and calls `print_grid`:



---

```
std::array<std::array<int, 4>, 3> grid = {{
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
}};
```

---

### Expected output:

```
1 2 3 4
5 6 7 8
9 10 11 12
```

## Part F — 2D `std::vector`

### Problem 1 — Trace the output

What does this program print?

#### Building a 2D vector

---

```
1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      std::vector<std::vector<int>>> frame;
6
7      frame.push_back({255, 128, 0});
8      frame.push_back({64, 32, 16});
9      frame.push_back({200, 200, 200});
10
11     std::cout << frame.size()    << std::endl; // line A
12     std::cout << frame[0].size() << std::endl; // line B
13     std::cout << frame[1][2]     << std::endl; // line C
14
15     frame[2][0] = frame[2][0] / 2;
16     std::cout << frame[2][0]     << std::endl; // line D
17
18     int total = 0;
19     for (int r = 0; r < frame.size(); r = r + 1)
20         for (int c = 0; c < frame[r].size(); c = c + 1)
21             total = total + frame[r][c];
22     std::cout << total           << std::endl; // line E
23
24     return 0;
25 }
```

---

Line A:

Line B:

Line C:

Line D:

Line E:

## Problem 2 — Spot the bug

This function should print every element of a 2D vector, row by row. It contains bugs.

### Buggy 2D print

---

```
1 #include <iostream>
2 #include <vector>
3
4 void print_image(const std::vector<std::vector<int>>& img) {
5     for (int row = 0; row < img.size(); row = row + 1) {
6         for (int col = 0; col <= img[row].size(); col = col + 1) {
7             std::cout << img[col][row] << " ";
8         }
9         std::cout << std::endl;
10    }
11 }
```

---

### Bugs:

Use : <https://godbolt.org/z/Kehxn6qe8>

Url of fix:

## Problem 3 — Write the code

Write a function:

---

```
std::vector<std::vector<int>> make_gradient(int rows, int cols);
```

---

that returns a 2D vector of size `rows × cols` where each element equals `row * cols + col` (the flat index of that element).

Then write a `main` that calls `make_gradient(3, 4)` and prints the result row by row, values separated by spaces.

### Expected output:

```
0 1 2 3
4 5 6 7
8 9 10 11
```

## Part G — Mixing everything

### Problem 1 — Nested loop with condition

Write a function:

---

```
int count_bright(const std::vector<std::vector<int>>& image, int threshold);
```

---

that counts how many pixels in the 2D image are strictly above `threshold`.

Then write a `main` that builds the following image, calls `count_bright` with a threshold of 100, and prints the result.

---

```
std::vector<std::vector<int>> image = {
    {50, 200, 30},
    {180, 90, 240},
    {10, 130, 70}
};
```

---

### Expected output:

### Problem 2 — Command-line image builder (harder)

Write a complete program invoked as:

---

```
./build <rows> <cols> <fill>
```

---

It must:

1. Validate that exactly 3 arguments are provided; otherwise print a usage message and return 1.
2. Read `rows`, `cols`, and `fill` using `atoi`.
3. Build a `std::vector<std::vector<int>>` of the given dimensions where every element equals `fill`.
4. Print the grid row by row, values separated by spaces.
5. Print the total sum of all elements on a final line.

**Expected output for `./build 2 3 50`:**

50 50 50

50 50 50

300